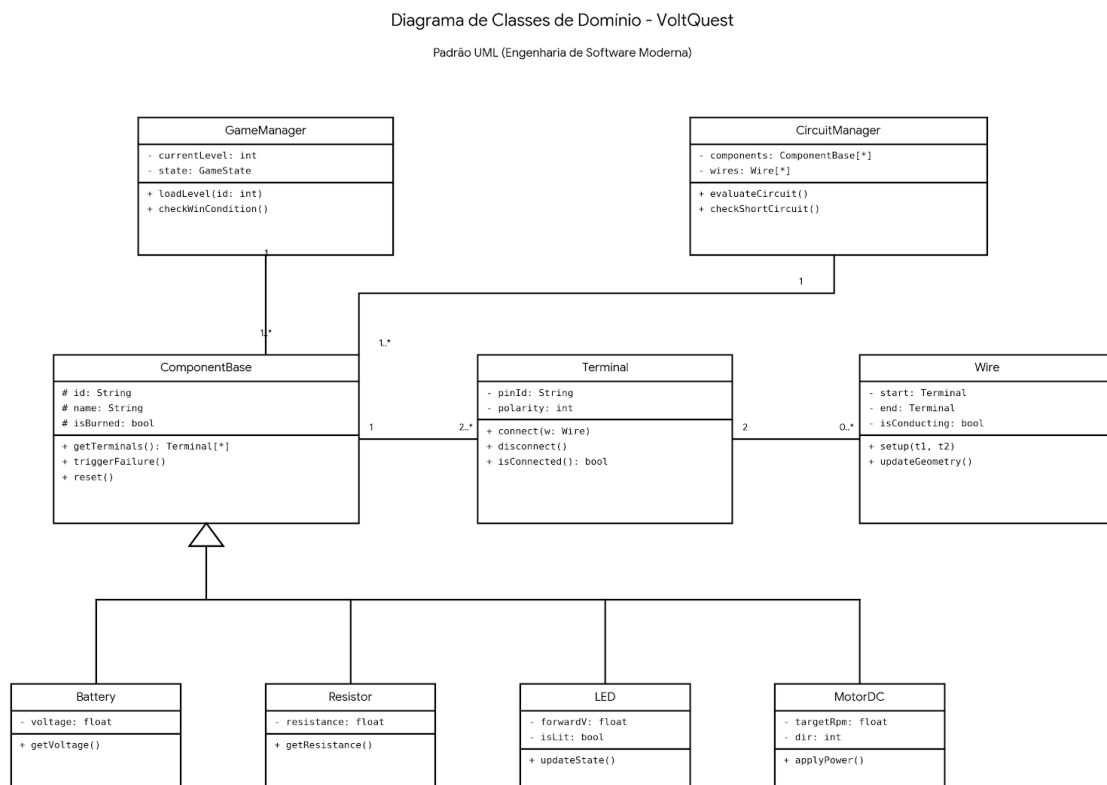


Modelo: Diagrama de classes

Desenvolvida com o suporte do Gemini, segue em anexo o diagrama de classes do jogo VoltQuest.

Figura 3 - Diagrama de classes



1. Os Controladores Globais (*Singletons*)

GameManager (O Diretor do Jogo)

- O que faz: Controla as regras de videogame (pontuação, fases, fluxo de telas).
- Atributos:
 - - **currentLevel: int**: Guarda o ID da fase atual (ex: 1 para "Acender LED", 4 para "Seguidor de linha").

- - **state: GameState**: Diz o que o jogador está fazendo agora (ex: *Editando, Rodando Teste, Falhou, Venceu*).
- Métodos:
 - + **loadLevel(id)**: Limpa a tela e carrega as peças e objetivos da próxima fase.
 - + **checkWinCondition()**: É chamado toda vez que o circuito muda. Verifica se o robô chegou no final da pista ou se a luz acendeu, declarando a vitória.

CircuitManager (O Motor Físico/Elétrico)

- O que faz: É o coração da simulação. Ele olha para as peças na tela como se fossem um Grafo (matemática discreta), onde os **Terminais** são os nós e os **Wires** (Fios) são as arestas.
- Atributos:
 - - **components: ComponentBase[*]**: Uma lista (array) contendo todas as peças que estão na bancada naquele momento.
 - - **wires: Wire[*]**: Uma lista de todos os fios desenhados.
- Métodos:
 - + **evaluateCircuit()**: Varre a malha. Calcula as resistências equivalentes e correntes com base nas Leis de Kirchhoff. (laço de repetição (**while** ou **for**) que percorra as conexões (fio -> pino -> componente -> pino -> fio) e some os valores.)
 - + **checkShortCircuit()**: Varredura de segurança. Se a corrente tender ao infinito (bateria ligada direto no GND sem resistência), ele detecta e manda o aviso de queima.

2. A Estrutura Física (O Circuito)

Representam o mundo físico da bancada.

ComponentBase (A Classe Mãe)

- O que faz: É um modelo genérico. Você nunca cria um "ComponentBase" no jogo; você cria LEDs, motores, etc. Mas todos eles herdam essas características básicas para que o **CircuitManager** possa tratar todos de forma padronizada (Polimorfismo).
- Atributos:
 - # **id**, # **name**: Identificadores únicos da peça.
 - # **isBurned: bool**: Sinaliza se a peça estourou por erro do jogador.
- Métodos:

- `+ getTerminals()`: Devolve a lista de pinos metálicos daquela peça.
- `+ triggerFailure()`: Toca o som de explosão, solta fumaça na tela e muda a imagem da peça para torrada.
- `+ reset()`: Restaura o componente para seu estado original

Terminal (Os Pinos de Conexão)

- O que faz: Representa os furos da protoboard, as pernas do LED ou os bornes do motor.
- Atributos:
 - `- pinId: String`: Ex: "Anodo", "Catodo", "VCC", "GND".
 - `- polarity: int`: Pode restringir o pino (ex: um pino que tem que ser positivo).
- Métodos:
 - `+ connect(Wire) / + disconnect()`: Associa ou desassocia um fio ao pino.
 - `+ isConnected()`: Responde se tem algo ligado ali.

Wire (O Fio Condutor)

O que faz: A ponte visual e lógica entre dois Terminais.

Atributos:

- `- start, - end`: Terminais de origem e destino do fio.
- `- isConducting: bool`: Define o estado visual (brilhante/verde se conduzindo corrente, opaco se aberto).

Métodos:

- `+ setup(t1, t2)`: Crava o fio entre dois pinos ao soltar o clique do mouse.
- `+ updateGeometry()`: Atualiza os vetores da linha visual caso o jogador mova o componente pela mesa.

3. As Especializações (Herança)

As classes na parte de baixo do diagrama são as peças reais. Elas já têm tudo que o `ComponentBase` tem, mas adicionam dados específicos de engenharia elétrica:

- Battery/ Bateria (Fonte de Tensão):
 - Tem o atributo de tensão (`voltage`).
 - O método `getVoltage()` é consumido pelo `CircuitManager` para iniciar o cálculo da malha.
- Resistor (Carga Passiva):

- Guarda a resistência em Ohms (**resistance**).
- LED (Diodo Emissor):
 - Possui a tensão de queda direta (**forwardV**) e um estado visual (**isLit**).
 - Seu método **updateState()** é chamado pelo simulador: se a corrente recebida for suficiente, ele acende a luz na tela. Se for excessiva, ele chama o **triggerFailure()** que herdou da mãe.
- MotorDC (Atuador Mecânico):
 - Traduz a elétrica para a robótica. Guarda a velocidade alvo (**targetRpm**) e a direção da rotação (**dir** = 1 para frente, -1 para trás).
 - **applyPower()**: Faz a roda do carrinho começar a girar na tela de testes dinâmicos (converte a energia para que as rodas girem na pista de testes).